HW 5: How Far Can You Throw?

Week 5 — functions, a list comprehension, and a search

Submission: paste the contents of `hw05.jl` into the Week 5 core CodeRunner question on Moodle. The program reads three numbers and prints two.

Core homework: find the best launch angle

A projectile leaves the origin with speed v_0 at an angle θ above the horizontal. Gravity pulls it down, and air drag pushes opposite to its velocity. Without drag, the farthest throw is at 45° and is easy to write on paper. Once drag matters, the best angle is no longer obvious, and the honest approach is to compute the range for every angle and look for the largest one.

Physics box: projectile with quadratic drag

Take $+x$ to the right and $+y$ upward, with the projectile launched from the origin over flat ground at $y = 0$. The speed is $v = \sqrt{v_x^2 + v_y^2}$, the launch speed is v_0 in m/s, the angle is θ in degrees, and the gravity magnitude is $g > 0$ in m/s^2. For a unit mass, quadratic drag gives

$$\frac{\mathrm{d}}{\mathrm{d}t}\begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix} = \begin{pmatrix} v_x \\ v_y \\ -b\,v\,v_x \\ -g - b\,v\,v_y \end{pmatrix},$$

where $b \geq 0$ has units of m^{-1} because it acts per unit mass. The drag term always opposes the velocity, so it can slow the projectile but never speed it up. Assuming a unit mass is a modeling choice: mass divides out of gravity, but it would not disappear from a force written this way for an arbitrary mass.

The model treats the projectile as a point, ignores spin and lift, assumes still air and flat ground, and uses a drag force proportional to speed squared. With $b = 0$ it reduces to the no-drag parabola, whose range is $v_0^2 \sin(2\theta)/g$ and whose maximum is at 45°. Real throws over long distances feel air density, spin, wind, and a changing drag coefficient, so treat the numbers below as a model result rather than a measured distance.

The two functions

Write these two functions in `hw05.jl`, keeping the names, argument order, and keyword names exactly:

```
function projectile_rhs(state, b, g)
    # state is [x, y, vx, vy]; return [dx/dt, dy/dt, dvx/dt, dvy/dt]
end

function projectile_range(v0, angle_deg; g = 9.81, b = 0.0, dt = 0.001)
    # Euler-integrate from the origin until the projectile hits the
   ground.
    # Return the horizontal distance x reached at landing.
end
```

Their jobs are:

- `projectile_rhs` takes the four-entry state vector and returns a four-entry vector holding the right-hand side above. It must not change the incoming vector.

- `projectile_range` builds the initial state

$$(0,\ 0,\ v_0 \cos\theta,\ v_0 \sin\theta),$$

integrates it with Euler's method,

$$\text{state}_{\text{next}} = \text{state}_{\text{now}} + \Delta t \cdot \texttt{projectile_rhs}(\text{state}_{\text{now}}, b, g),$$

and stops when the height first becomes negative. Return the horizontal coordinate from the last step with $y \geq 0$.

Write the rate equations once, inside `projectile_rhs`, and let `projectile_range` call that function. The point of the exercise is to keep the physics in one place and reuse it. Assume the inputs are sensible: a positive speed, a nonnegative drag coefficient, positive gravity, and angles from $1°$ to $89°$. No error checking is required.

Why Euler is enough here

We are learning to organize a calculation, not to win an accuracy contest. Euler's method takes a small step along the current derivative, then recomputes the derivative at the new state. With `dt = 0.001` s the curve is close enough for finding which angle wins, and the CodeRunner tolerances below allow for differences between reasonable step sizes. If you later care about precise numbers, smaller steps or a better integrator are the natural next questions.

Search the angles with a list comprehension

Use a list comprehension to compute all the ranges at once, then let `argmax` tell you the position of the largest entry:

```
angles = 1:89
ranges = [projectile_range(v0, angle; g = g, b = b) for angle in angles]
best_index = argmax(ranges)
println(angles[best_index])
println(round(ranges[best_index]; digits = 2))
```

`argmax(ranges)` returns the index of the largest value, so `angles[best_index]` is the winning angle. The complete program reads its three values from one line of standard input:

```
values = parse.(Float64, split(readline()))
v0, b, g = values[1], values[2], values[3]
angles = 1:89
ranges = [projectile_range(v0, angle; g = g, b = b) for angle in angles]
best_index = argmax(ranges)
println(angles[best_index])
println(round(ranges[best_index]; digits = 2))
```

Print nothing else: no prompts, headings, or extra numbers.

Values to check against

Work through these before submitting. The right-hand side is immediate: for `state = [0.0, 0.0, 10.0, 10.0]`, $b = 0.1$, $g = 10$, the speed is $\sqrt{200}$, so both velocity derivatives equal $-0.1\sqrt{200} \cdot 10 = -10\sqrt{2}$:

```
println(projectile_rhs([0.0, 0.0, 10.0, 10.0], 0.1, 10.0))
```

```
[10.0, 10.0, -14.142135623730951, -24.14213562373095]
```

The table below gives ranges computed with `dt = 0.001`. Small changes from a different step size are expected and acceptable.

v_0 (m/s)	b (m^{-1})	best angle (deg)	best range (m)	at 45° (m)
20	0	45	40.79	40.79
20	0.001	45	39.52	39.52
20	0.01	43	31.39	31.33

The final column shows why the search matters. With weak drag the best angle is still $45°$, and the range there equals the best range. With stronger drag, $45°$ is slightly past the optimum and the winner moves down to about $43°$.

For the assignment's own input, print the angle and then the range, each on its own line. For example, `hw05.jl` run with the line

```
20 0.001 9.81
```

must print

```
45
39.52
```

The grader compares the range with a 5% relative tolerance and the angle with a tolerance of $\pm 1°$. That allowance covers step-size differences; the physics and the search still have to be right.

Submission

Submit `hw05.jl` containing the two function definitions and the short program that reads the three values and prints the best angle and best range. Keep tests, plotting, and extra demonstration calls out of the file; the grader runs the program with its own input line. No figure or written report is required.

The separate optional bonus treats the double pendulum. It is worth one-third of a final-grade point, is not a prerequisite for this homework or later work, and nothing that appears only there is examined.